



Guía de Errores en Transfer Learning

Fallos comunes, síntomas en las curvas de aprendizaje y estrategias de corrección



En Transfer Learning, muchos problemas no proceden de la arquitectura en sí, sino de cómo se descongelan las capas, cómo se ajustan los hiperparámetros y cómo se prepara el flujo de datos.

Error 1 — Catastrophic Forgetting (Olvido Catastrófico)



Causa

Descongelar demasiadas capas a la vez o usar un Learning Rate demasiado alto en Fine-Tuning. Las capas base pierden los patrones visuales previamente aprendidos.



Síntoma

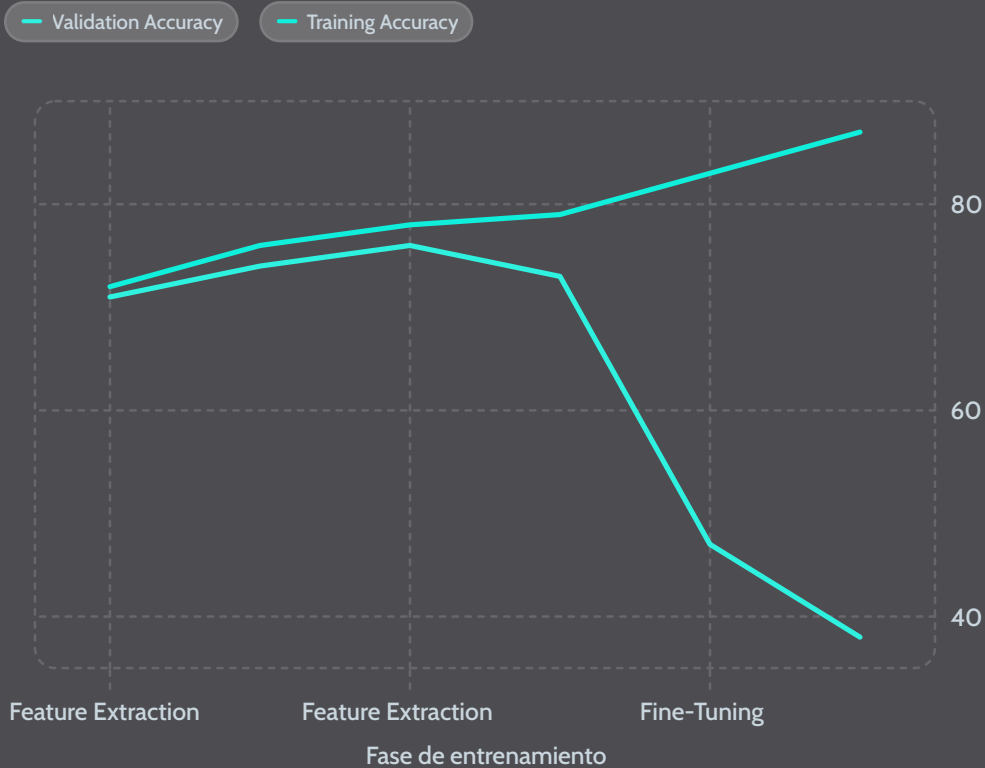
Caída **repentina y drástica** de la precisión en validación justo al iniciar el Fine-Tuning. La validación colapsa mientras el training puede seguir mejorando.



Solución

- Usar Learning Rate muy bajo: `1e-5`
- Descongelar capas de forma **progresiva** (de arriba hacia abajo)
- Monitorizar `val_accuracy` en cada época tras descongelar

Curva conceptual — Señal de alerta



La caída brusca de **Validation Accuracy** tras iniciar el Fine-Tuning es la señal característica del Catastrophic Forgetting.

Error 2 — Riesgo de Overfitting con Datasets Pequeños



Causa

Aplicar Fine-Tuning agresivo cuando el dataset nuevo no tiene suficientes muestras. El modelo memoriza el training en lugar de generalizar.



Síntoma

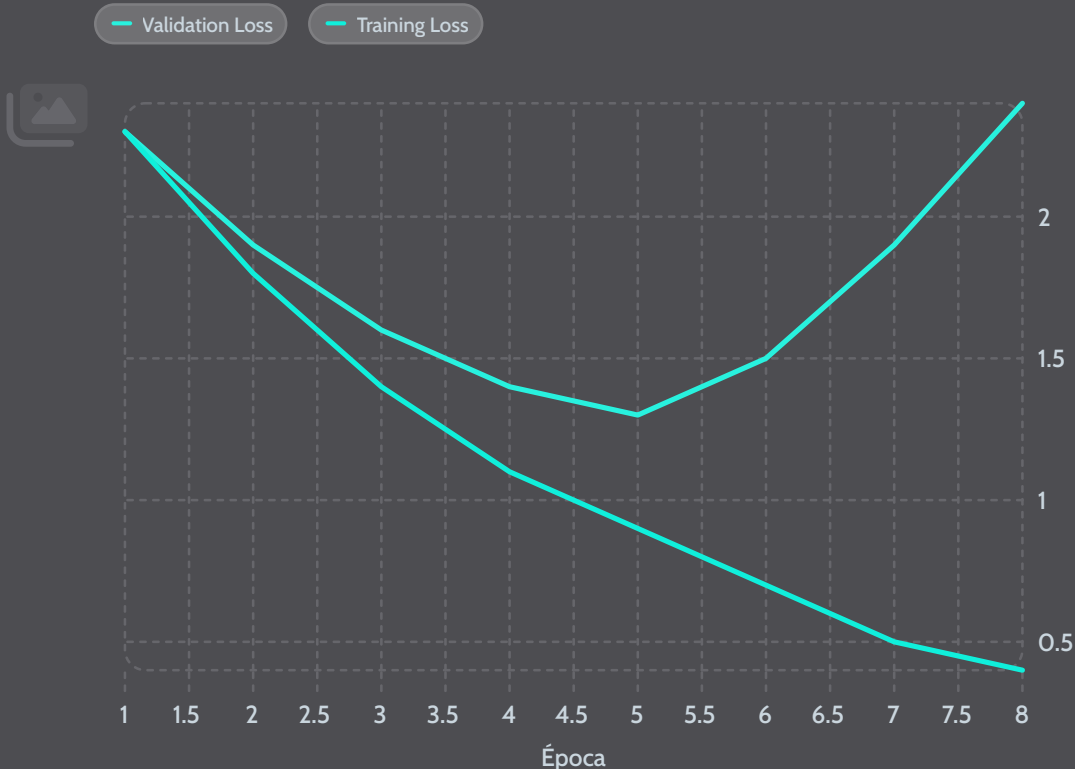
Divergencia clara: Training Loss sigue bajando, pero Validation Loss empieza a subir. Brecha creciente entre ambas curvas.



Solución

- Limitarse a **Feature Extraction** si el dataset es muy pequeño
- Añadir `Dropout(0.5)` antes de la capa final
- Aplicar **Data Augmentation** agresivo
- Usar `EarlyStopping` monitorizando `val_loss`

Curva conceptual — Zona de Overfitting



Training Loss ↓ continuo + Validation Loss ↑ = **Overfitting**. El modelo memoriza el training y pierde capacidad de generalización.

Error 3 — Incompatibilidad de Normalización / Preprocesamiento



Causa

No aplicar la función `preprocess_input` propia de la red base. Pasar imágenes en rango `[0, 255]` cuando la red espera `[-1, 1]` o valores estandarizados por ImageNet.



Síntoma

La red **no converge desde las primeras épocas** o presenta un rendimiento extremadamente bajo sin causa aparente en la arquitectura.



Solución

Utilizar **siempre** la función de preprocesamiento del modelo base:

```
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input

x = preprocess_input(x) # Aplica media y std de ImageNet
```

Flujo correcto vs. incorrecto



Flujo correcto

Imagen → `preprocess_input` → Modelo base → Clasificador → Predicción fiable



Flujo incorrecto

Imagen `[0,255]` → Sin preprocesamiento → Modelo base → Distribución de entrada errónea → Rendimiento deficiente



Cada modelo base (VGG16, ResNet50, EfficientNet, MobileNet) tiene su propia `preprocess_input`. No son intercambiables entre sí.

Errores Técnicos de Implementación

Error 4 — Mal Uso del Modo Inferencia en BatchNormalization



Causa

Entrenar la base sin pasar `training=False` en el modelo base al construir la red funcional. BatchNormalization actualiza sus estadísticas de forma incorrecta.



Síntoma

Inestabilidad severa en las métricas durante el entrenamiento o resultados muy pobres al evaluar en test.



Solución

Fijar explícitamente `training=False` al llamar la base:

```
inputs = tf.keras.Input(shape=(224, 224, 3))
x = base_model(inputs, training=False)
x = tf.keras.layers.GlobalAveragePooling2D()(x)
outputs = tf.keras.layers.Dense(num_classes,
activation='softmax')(x)
model = tf.keras.Model(inputs, outputs)
```

¿Por qué `training=False`?

`training=True` (por defecto)

BatchNorm usa las estadísticas del batch actual y actualiza las medias/varianzas móviles. En la base congelada esto corrompe las estadísticas aprendidas en ImageNet.

`training=False` (correcto)

BatchNorm usa las estadísticas móviles ya aprendidas durante el pre-entrenamiento. La base se comporta como un extractor de características estable y consistente.



Este error es especialmente silencioso: el código ejecuta sin errores, pero el rendimiento es inexplicablemente bajo.

Error 5 — Olvidar Recompilar tras Cambiar trainable



Causa

Modificar el estado de congelado de las capas (`.trainable = True`) y llamar a `fit()` directamente sin recompilar. Keras no detecta automáticamente el cambio en los parámetros entrenables.



Síntoma

El entrenamiento continúa como si **no se hubieran descongelado las capas**. No se refleja ningún cambio en el aprendizaje ni en las métricas.



Solución

Recompilar siempre con `model.compile()` inmediatamente después de cambiar `.trainable`:

```
base_model.trainable = True

model.compile(

optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
loss='categorical_crossentropy',
metrics=['accuracy']
)
```

Punto clave

El paso de recompilación es obligatorio. Sin él, Keras mantiene el grafo de entrenamiento anterior ignorando las capas recién descongeladas.



Usar siempre un Learning Rate más bajo en Fine-Tuning: `1e-5` en lugar de `1e-3`.

HOJA DE RESUMEN Y BUENAS PRÁCTICAS

Diagnóstico rápido y reglas esenciales antes de entrenar

DIAGNÓSTICO RÁPIDO

PROBLEMA	SEÑAL PRINCIPAL	PRIMERA ACCIÓN
Catastrophic Forgetting	Caída brusca tras Fine-Tuning	Reducir Learning Rate y descongelar progresivamente
Overfitting	Training loss ↓ / Validation loss ↑	Feature Extraction + Dropout + Data Augmentation
Preprocesamiento incorrecto	No converge desde el inicio	Revisar <code>preprocess_input</code>
BatchNormalization	Métricas inestables en test	Usar <code>training=False</code>
trainable sin recompilar	No se refleja el Fine-Tuning	Ejecutar <code>model.compile()</code>

4 REGLAS DE ORO

Regla de oro 1

Empieza con Feature Extraction antes de hacer Fine-Tuning. Esto preserva los patrones aprendidos en ImageNet y reduce el riesgo de Catastrophic Forgetting.

Regla de oro 2

Usa un Learning Rate bajo durante Fine-Tuning (1e-5 o menor). Cambios grandes en los pesos pueden destruir el conocimiento pre-entrenado.

Regla de oro 3

Después de modificar `.trainable`, recompila siempre con `model.compile()`. Keras no detecta automáticamente los cambios en los parámetros entrenables.

Regla de oro 4

Controla las curvas de validación para detectar Overfitting y otros problemas de entrenamiento. Las métricas son tu mejor diagnóstico.

Diagnosticar correctamente el problema antes de ajustar el modelo evita entrenamientos innecesarios y reduce el riesgo de sobreajuste.